

TA-BRPL: Trust-Aware Backpressure RPL

Blackhole + Sinkhole 복합 공격 환경에서의
온-노드 신뢰 모델 설계, 구현 및 검증

연구 노트 v0.3.6 (최종 수정: 2026-03-24)

TA-BRPL Research Group
Contiki-NG / Cooja 기반 구현

March 24, 2026

Contents

1	연구 목표와 설계 철학	3
1.1	연구 동기	3
1.2	설계 목표	3
1.3	설계 원칙	4
2	전체 시스템 아키텍처	4
2.1	계층 구조	4
2.2	정보 흐름	4
2.3	파라미터 단일 소스 정책	5
3	신뢰 성분 상세 설명	5
3.1	T_{fwd} : 포워딩 신뢰 (Forwarding Trust)	5
3.1.1	에코 기반 end-to-end 측정 원리	6
3.1.2	PRR 폴백: 부트스트래핑 메커니즘	7
3.1.3	카운터 감쇠: 할빙(halving) 방식	8
3.1.4	T_{fwd} 날카로움(sharpening)	8
3.2	T_{ctrl} : 제어 평면 신뢰 (Control-Plane Trust)	8
3.2.1	Rank 이상 지표 (A_{rank})	8
3.2.2	DIO 빈도 이상 지표 (A_{DIO})	8
3.2.3	Version 이상 지표 (A_{ver})	8
3.2.4	가중합 집계	9
3.3	T_{hon} : 혼잡 정직성 신뢰 (Congestion-Honesty Trust)	9
3.4	가중 기하평균 집계	10
3.5	비대칭 EWMA 업데이트	10
3.6	독립 T_{fwd} EWMA: <code>trust_fwd</code>	11
4	검증 모델 (Validation Model)	11
4.1	검증 상태 (Review State)	11
4.2	리뷰 스코어 시스템	12
4.3	상태 전환 조건	12
4.4	검증 패널티 배율	13

5	임계값 정책, 패널티 및 격리 메커니즘	13
5.1	3단계 임계값 정책	13
5.2	동적 패널티 배율	14
5.3	Escape 메커니즘	15
5.4	격리 및 해제 메커니즘	16
6	BRPL 인터페이스 상세	17
6.0.1	brpl_trust_get(node_id)	17
6.0.2	brpl_penalty_scale_get(node_id)	18
6.0.3	brpl_validation_penalty_scale_get(node_id)	18
6.0.4	brpl_escape_mode_get(node_id)	18
6.0.5	brpl_trust_parent_allowed(node_id)	18
7	Parent 선택 안정화	18
8	SMTrust 구조적 한계 분석	19
8.1	SMTrust 개요	19
8.2	TI Floor 문제	19
9	실험 설정	20
9.1	토폴로지와 노드 역할	20
9.2	실험 타임라인	22
9.3	통제된 랜덤 토폴로지 실험 설계 (구현 완료, 미실행)	22
9.4	비교 프로토콜	24
10	실험 결과	24
10.1	30-Seed 기본 결과	24
10.2	오탐 실험 (No-Attack Baseline)	25
10.3	혼잡-공격 분리 실험 (V3)	26
10.4	EWMA λ 민감도 분석 (V5)	26
10.5	임계값 민감도 분석 (V6)	27
10.6	성분 절제 실험	27
10.7	손실률 로버스트니스	27
11	분석 도구 버그 수정	28
11.1	버그 1: CSV,RX 오프셋 오류	28
11.2	버그 2: Seq-Only PDR 매칭	28
12	전체 파라미터 목록	29
13	구현상 한계 및 향후 과제	29
14	결론	30

Abstract

TA-BRPL(Trust-Aware Backpressure RPL)은 저전력 손실 네트워크(LLN)에서 backpressure 기반 혼잡 인식 라우팅과 온-노드 신뢰 관리를 결합한 프로토콜이다. 기반이 되는 BRPL(Backpressure RPL)의 목적 함수에 세 가지 신뢰 성분, 즉 T_{fwd} (포워딩 신뢰), T_{ctrl} (제어 평면 신뢰), T_{hon} (혼잡 정직성 신뢰)을 직접 결합하며, 별도의 2단계 검증 모델이 하드 격리(blacklist) 결정을 독립적으로 담당한다.

실험 환경은 Contiki-NG/Cooja 시뮬레이터 기반의 6×6 GRID 토폴로지이며, blackhole 공격자 3개(node 2, 3, 4)와 sinkhole 공격자 1개(node 18)가 동시에 동작하는 복합 공격 시나리오이다. 전체 30개 random seed 실험에서 TA-BRPL은 공격 구간(350–650초) PDR **0.8892**와 회복 구간(650–900초) PDR **0.8809**를 달성하였다. 동일 조건의 비교군: RPL 0.8726/0.8690, BRPL 0.8820/0.8702, SMTrust 0.8693/0.8610(유효 21 seed)이며, 무공격 환경에서 TA-BRPL의 과오탐(false positive)은 관찰되지 않았다.

본 문서는 TA-BRPL의 모든 구성 요소, 설계 의도, 구현 세부 사항, 동작 원리, 실험 설정 및 결과를 한국어로 총정리한 기술 참조 문서이다. 본 버전의 정량 결과는 고정 GRID 실험에 한정되며, 통제된 랜덤 토폴로지 분포 실험은 실행 프레임워크 구현까지 완료된 상태이다(미실행).

키워드: IoT, LLN, RPL, BRPL, 신뢰 관리, Blackhole 공격, Sinkhole 공격, 선택적 포워딩, Cooja, Contiki-NG

1 연구 목표와 설계 철학

1.1 연구 동기

LLN 환경에서의 라우팅 공격은 두 가지 형태로 대별된다. 첫째, **Blackhole 공격**은 노드가 라우팅 프로토콜에 정상 참여하면서 IP 계층에서 전달해야 할 패킷을 선택적으로 드롭한다. MAC 계층의 ACK는 정상 전달되므로 상위 계층에서는 탐지가 어렵다. 둘째, **Sinkhole 공격**은 노드가 DIO(DODAG Information Object) 메시지의 rank 필드를 비정상적으로 낮게 광고하여 인근 노드들을 자신을 통해 라우팅하도록 유인하고, 유입된 트래픽을 드롭하는 방식이다.

기존 신뢰 관리 접근법의 한계는 크게 두 가지다.

1. **forwarding rate 기반 감시의 한계:** 혼잡으로 패킷을 드롭하는 정직한 노드와 의도적으로 드롭하는 공격자를 forwarding rate만으로는 구분할 수 없다.
2. **Trust floor 문제:** 가중합 기반 trust 모델에서 일부 성분이 환경 상수로 고정되면, trust의 최솟값이 탐지 임계값 위에 묶여 공격자가 배제되지 않는다. (SMTrust의 구조적 한계, Section 8 참조)

1.2 설계 목표

TA-BRPL은 다음 목표를 동시에 만족하도록 설계되었다.

- **G1:** BRPL이 제공하는 backpressure 기반 혼잡 인식 라우팅 능력을 유지한다.
- **G2:** Blackhole 및 sinkhole에 대해 노드 내 자율적 탐지 및 우회 능력을 제공한다.
- **G3:** 정상 노드에 대한 과오탐(false positive)과 과패널티를 최소화한다.
- **G4:** 기존 RPL-Classic 소스를 직접 수정하지 않고, weak-symbol 오버라이드 방식으로 결합하여 모듈성을 확보한다.

1.3 설계 원칙

- **에코 기반 end-to-end 확인**: 단순 도청(overhearing)이 아닌 root의 에코 응답을 통해 패킷이 실제로 root까지 도달했는지 확인한다.
- **혼잡과 공격의 분리**: T_{hon} 을 통해 혼잡으로 인한 forwarding 저하와 의도적 드롭을 구분한다.
- **비대칭 EWMA**: 공격 탐지 반응은 빠르게, 신뢰 회복은 느리게 설정하여 on-off 공격 전략을 억제한다.
- **2단계 검증**: trust EWMA와 독립적인 누적 증거 기반 검증 모델로 하드 격리 결정을 보수적으로 내린다.

2 전체 시스템 아키텍처

2.1 계층 구조

TA-BRPL은 Contiki-NG의 RPL-Classic 라우팅 스택 위에서 동작한다. 전체 소프트웨어 계층 구조는 다음과 같다.

사용자 프로세스 (sender.c, receiver_root.c)
TA-BRPL 신뢰 엔진 (ta-brpl-trust.c/h)
BRPL 목적 함수 (rpl-brpl.c) ← weak-symbol 오버라이드
RPL-Classic 라우팅 스택 (Contiki-NG)
IPv6/UDP/MAC 스택

핵심 구현 파일은 ta-brpl-trust.c/h이며, BRPL의 목적 함수(rpl-brpl.c)가 trust 정보를 요청할 때 C언어의 weak-symbol 메커니즘을 통해 TA-BRPL의 구현이 자동으로 호출된다. 이 설계 덕분에 TA-BRPL 모듈을 컴파일에서 제외하더라도 RPL-Classic 및 BRPL의 원본 소스를 수정할 필요가 없다.

2.2 정보 흐름

trust 엔진으로 입력되는 이벤트는 다음 세 경로를 통해 전달된다.

1. **에코 응답 후 (echo_rx_callback)**: sender.c에서 root로 UDP 패킷을 전송할 때 현재 preferred parent를 remember_tx_parent(seq, parent_id)로 기록한다. root가 패킷을 수신하면 에코 응답을 발신자에게 반환하고, 에코 수신 시 take_tx_parent(seq)로 해당 parent를 역추적하여 ta_trust_notify_forwarded(parent_id)를 호출한다. 이를 통해 에코 기반 포워딩 관찰값 F_{ij} 를 갱신한다.
2. **IP 입력 후 (ta_ip_input_hook)**: Contiki-NG의 NETSTACK_IP_PACKET_PROCESSOR 후를 통해 수신된 패킷 중 RPL DIO 메시지(ICMPv6, code=0x02)를 파싱하여 T_{ctrl} 지표(rank/version/frequency 이상)를 갱신한다.
3. **BRPL 큐 콜백**: RPL parent table에서 이웃 노드의 BRPL 큐 광고값(Q_{adv} , Q_{max})을 주기적으로 읽어 T_{hon} 계산에 사용한다.

fig_arch: T_fwd Echo-Based Measurement Pipeline

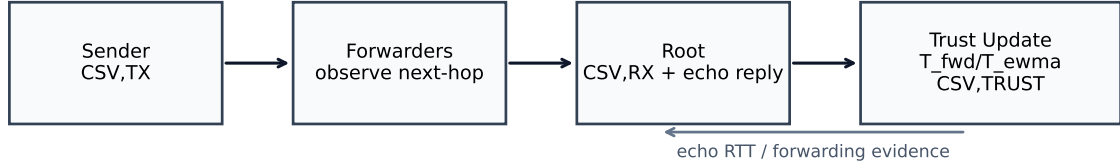


Figure 1: TA-BRPL 시스템 정보 흐름도. sender가 패킷을 전송하면 parent ID가 기록되고, root의 에코 응답 수신 시 해당 parent의 T_{fwd} 가 갱신된다. DIO 수신은 IP 입력 혹에서 파싱되어 T_{ctrl} 에 반영된다.

4. **Parent 교체 콜백 (RPL_CALLBACK_PARENT_SWITCH):** `project-conf.h`에서 `RPL_CALLBACK_PARENT_SWITCH`를 `brpl_parent_switch_callback`으로 정의하여, preferred parent 교체 시 `brpl_preferred_parent` 호출된다. 이를 통해 `current_parent_id`와 `escape` 타이머를 갱신한다. 이 혹이 없으면 `escape` 메커니즘은 발동하지 않는다.

trust 엔진에서 BRPL 목적 함수로 제공하는 출력 인터페이스:

- `brpl_trust_get(node_id)`: trust EWMA 값 $[0, 1000]$ 반환
- `brpl_penalty_scale_get(node_id)`: 신뢰 기반 동적 패널티 배율 반환
- `brpl_validation_penalty_scale_get(node_id)`: 검증 모델 기반 패널티 배율 반환
- `brpl_escape_mode_get(node_id)`: escape 모드 활성화 여부 반환
- `brpl_trust_parent_allowed(node_id)`: parent 후보 허용 여부 반환

2.3 파라미터 단일 소스 정책

모든 tunable 파라미터는 `project-conf.h`의 C 매크로로 정의된다. 각 프로토콜 변형의 `Makefile.*`은 모드 플래그(`TABRPL_MODE=1` 등)만 설정하며, 임계값, λ 값, 패널티 배율 등은 모두 `project-conf.h`에서 집중 관리된다. 민감도 실험은 컴파일러 커맨드라인에서 특정 매크로만 오버라이드하는 방식으로 수행된다.

3 신뢰 성분 상세 설명

3.1 T_{fwd} : 포워딩 신뢰 (Forwarding Trust)

$T_{\text{fwd}}(i, j)$ 는 노드 i 의 관점에서 이웃 j 가 패킷을 신뢰성 있게 root까지 전달하는지를 측정하는 성분이다.

3.1.1 에코 기반 end-to-end 측정 원리

왜 도청(overhearing)이 아닌 에코 기반인가? 초기 버전에서는 IEEE 802.15.4 채널에서 이웃 노드가 다음 홉으로 패킷을 전달하는 순간을 수동으로 도청(passive overhearing)하여 T_{fwd} 를 측정하였다. 그러나 802.15.4 unicast 통신에서 CSMA 드라이버는 자신에게 주소되지 않은 프레임을 MAC 계층에서 폐기한다. 따라서 진정한 상향 포워딩 이벤트를 도청하는 것은 물리적으로 불가능하다. 더 나아가, 하향 경로(root $\rightarrow$ sender)의 에코 응답이 parent를 거쳐 오는 경우 이를 포워딩 증거로 잘못 인식하면 선택적 포워딩 공격자에게 거짓 양성 포워딩 점수를 부여할 수 있다.

에코 기반 방식은 이 두 문제를 모두 해결한다. 패킷이 실제로 root까지 도달했는지를 root의 에코 응답으로 확인하므로, IP 계층 선택적 드롭을 MAC 투명하게 탐지할 수 있다.

동작 흐름

1. 노드 i 가 이웃 j 를 preferred parent로 하여 데이터 패킷(seq= s)을 전송할 때, ta_trust_notify_sent를 호출하여 카운터 S_{ij} 를 1 증가시킨다. 동시에 remember_tx_parent(s, j)로 seq-parent 쌍을 환형 버퍼에 기록한다.
2. Root가 해당 패킷을 수신하면 발신자에게 에코 응답("seq= s t0= t ")을 반환한다.
3. 발신자가 에코를 수신하면 echo_rx_callback이 호출된다. take_tx_parent(s)로 해당 seq를 전송했던 parent j 를 역추적하고, ta_trust_notify_forwarded(j)를 호출하여 F_{ij} 를 1 증가시킨다.

의사코드 (구현 참조: sender.c, ta-brpl-trust.c)

```
on_data_send(seq, parent_id):
    ta_trust_notify_sent(parent_id)    # S_ij++
    remember_tx_parent(seq, parent_id) # ring buffer

on_echo_receive(seq):
    parent_id = take_tx_parent(seq)
    if parent_id != 0:
        ta_trust_notify_forwarded(parent_id) # F_ij++

on_trust_window_update(neighbor j):
    if fwd_sent_new(j) == 0:
        tfwd = prr_from_etx_or_fallback(j)    # PRR fallback
    else:
        expected = fwd_sent_new(j) * prr_from_etx_or_fallback(j)
        tfwd = (fwd_observed_new(j) + alpha) / (expected + alpha + beta)
    tfwd = clamp_scale(tfwd, 0, 1000)
```

수식 Laplace smoothing 파라미터 (α, β) 를 적용한 포워딩 성공률:

$$T_{\text{fwd}}^{(\text{raw})}(i, j) = \frac{F_{ij} + \alpha}{E_{ij} + \alpha + \beta}, \quad E_{ij} = S_{ij} \cdot \hat{P}_{\text{RR}}(j) \quad (1)$$

여기서 E_{ij} 는 링크 손실을 고려한 기대 포워딩 횟수이고, $\hat{P}_{\text{RR}}(j)$ 는 MAC 계층 ETX로부터 추정된 패킷 수신률(PRR)이다. $\alpha = \beta = 1$ 로 Laplace 스무딩을 적용하여 초기 관찰에서 trust가 0 또는 1로 급격히 치우치는 것을 방지한다.

3.1.2 PRR 폴백: 부트스트래핑 메커니즘

에코 기반 측정의 근본적 한계는 에코 회신이 하향 경로(root → sender)의 수렴을 필요로 한다는 점이다. DODAG 형성 초기에는 DAO가 root에 도달하지 않아 하향 경로가 개통되지 않으며, 실측 에코 도달률은 약 7%에 불과하다.

이 상태에서 방치하면 다음 사망 나선(death spiral)이 발생한다.

$$\begin{aligned} S_{ij} &\nearrow (\text{패킷 전송마다 증가}) \\ F_{ij} &= 0 (\text{에코가 도달하지 않음}) \\ T_{\text{fwd}} &= \frac{0+1}{S_{ij}+2} \rightarrow 0 (\text{S 누적으로 붕괴}) \end{aligned}$$

trust EWMA가 $\tau_{\text{join}} = 350$ 미만으로 하락하면 모든 parent가 후보 집합에서 제외되어 라우팅이 완전히 붕괴된다.

이를 방지하기 위해, 현재 update window에서 새로운 전송이 없는 경우 (`fwd_sent_new == 0`) 링크 통계 ETX로부터 추정된 PRR을 T_{fwd} 의 대안 값으로 사용한다.

$$T_{\text{fwd}}(i, j) = \begin{cases} \hat{P}_{\text{RR}}(j) & \text{fwd_sent_new} = 0 \text{ (PRR 폴백)} \\ T_{\text{fwd}}^{(\text{raw})}(i, j) & \text{fwd_sent_new} > 0 \text{ (에코 기반)} \end{cases} \quad (2)$$

PRR 추정 방법 Contiki-NG의 link-stats 모듈이 관리하는 ETX(Expected Transmission Count)로부터:

$$\hat{P}_{\text{RR}}(j) = \text{clamp}\left(\frac{\text{LINK_STATS_ETX_DIVISOR}}{\text{ETX}_j}, P_{\text{min}}, P_{\text{max}}\right) \quad (3)$$

ETX 통계가 아직 충분하지 않은 경우(`link_stats_is_fresh` 미달)에는 $P_{\text{fallback}} = 1000$ (lossless 가정)을 사용한다.

fwd_sent_new 조건의 의미 `fwd_sent_new`는 현재 update window에서 새롭게 전송한 패킷 수이며, 매 업데이트 주기 후 0으로 리셋된다(하프닝 없음). 이를 조건으로 사용하는 이유는 다음 두 경우를 정확히 구분하기 위해서이다.

- **이번 창에 전송이 없는 경우** (`fwd_sent_new=0`): 현재 parent로 사용되지 않는 이웃이므로 에코 증거가 없는 것은 당연하다. PRR 폴백을 사용하여 통신 품질 기반 trust를 유지한다. 이 경우는 부트스트래핑뿐만 아니라 경로 변동(route churn) 이후 이전 parent가 일시적으로 비활성화된 경우도 포함된다.
- **이번 창에 전송이 있으나 에코가 없는 경우** (`fwd_sent_new > 0`, $F_{ij}^{(\text{new})} = 0$): 패킷이 전송되었으나 root에 도달하지 않은 전형적인 blackhole 시그니처이다. 에코 기반 수식을 적용하면 T_{fwd} 가 낮아져 공격자 탐지로 이어진다.

이전 구현에서 `fwd_observed_new == 0`을 조건으로 사용했을 때의 문제점: 공격자는 항상 `fwd_observed_new`가 0이었으므로 PRR 폴백을 받아 $T_{\text{fwd}} \approx 1000$ 을 유지하고, 오히려 정직한 노드(에코 70% 도달률)가 낮은 $T_{\text{fwd}} \approx 700$ 을 가지는 역전 현상이 발생하였다. `fwd_sent_new`를 조건으로 변경하여 이 문제를 해결하였다.

3.1.3 카운터 감쇠: 할빙(halving) 방식

매 업데이트 주기($\Delta t = 60$ 초)마다 누적 카운터를 우측 비트시프트로 반감한다.

$$S_{ij} \leftarrow S_{ij} \gg 1, \quad F_{ij} \leftarrow F_{ij} \gg 1 \quad (4)$$

완전 리셋(zeroing) 대신 할빙을 사용하는 이유는, 리셋 시 공격자가 잠시(한 window 동안) 정상 동작을 보이는 것만으로 T_{fwd} 가 다시 중립값(500)으로 초기화되어 trust가 회복되는 on-off 공격 취약점이 발생하기 때문이다. 할빙을 통해 과거 관찰의 영향이 지속적으로 감쇠하면서도 완전히 사라지지 않아, 약 2~3 업데이트 창 의 유효 메모리를 유지한다.

3.1.4 T_{fwd} 날카로움(sharpening)

옵션으로 TA_TFWD_SHARPEN_SCALE 파라미터를 1000 초과로 설정하면 T_{fwd} 를 중립값(500) 기준으로 확장하여 honest/attacker 간 차이를 증폭한다. 현재 기본값은 1000으로 비활성화 상태이다.

3.2 T_{ctrl} : 제어 평면 신뢰 (Control-Plane Trust)

T_{ctrl} 은 이웃 노드의 RPL 제어 메시지 동작에서 이상 징후를 탐지하는 성분이다. IP 입력 혹은 DIO ICMPv6 메시지(type=155, code=0x02)를 파싱하여 세 가지 이상 지표를 갱신한다.

3.2.1 Rank 이상 지표 (A_{rank})

- **절대 이상**: DIO에 포함된 rank가 root rank(= min_hoprankinc)보다 낮은 경우. 물리적으로 불가능한 값이므로 rank 이상 카운터를 즉시 증가시킨다. 이는 sinkhole이 rank=128(또는 더 낮은 값)을 광고하여 root보다 더 좋은 경로를 주장하는 공격을 탐지한다.
- **과도한 rank 감소**: 동일한 DODAG version 번호를 유지하면서 rank가 min_hoprankinc / 2 이상 감소하는 경우. 실제 링크 품질 개선으로 인한 rank 변동(ETX 기반)과 구별하기 위해 hysteresis 임계값(min_hoprankinc / 2)을 적용한다.

3.2.2 DIO 빈도 이상 지표 (A_{DIO})

Trickle 타이머 기반의 정상 DIO 전송률(TA_TRUST_DIO_NORMAL_RATE = 5회/창) 대비 과도한 DIO 전송은 DIO flooding 또는 trickle 억제 공격의 신호로 해석한다.

$$A_{\text{DIO}} = \frac{\max(0, \text{ctrl_dio_count} - N_{\text{normal}})}{\text{ctrl_dio_count}} \quad (5)$$

3.2.3 Version 이상 지표 (A_{ver})

Root에서 전파되지 않은 RPL DODAG version 번호의 증가는 강제 DODAG 재시작을 유발하는 version attack의 신호이다. 동일 version 맥락에서 rank가 급감하면 rank 이상으로 처리하고, version이 변경되면 새 baseline으로 업데이트하여 합법적인 DODAG 재시작을 허용한다.

3.2.4 가중합 집계

$$A_{ij} = \frac{w_1 \cdot A_{\text{rank}} + w_2 \cdot A_{\text{DIO}} + w_3 \cdot A_{\text{ver}}}{10} \quad (6)$$

$$T_{\text{ctrl}}(i, j) = 1 - A_{ij} \quad (7)$$

기본 가중치: $w_1 = 5$ (rank), $w_2 = 3$ (DIO), $w_3 = 2$ (version).

카운터는 매 업데이트 주기마다 할빙되어 과거 이상의 영향이 시간에 따라 감쇠한다. 데이터가 없는 경우($\text{ctrl_dio_count} = 0$) $A_x = 0$ 을 적용하여 $T_{\text{ctrl}} = 1.0$ 을 유지한다.

의사코드 (구현 참조: ta_ip_input_hook)

```
on_dio_rx(neighbor j, rank, version):
    ctrl_dio_count[j] += 1

    if rank < root_rank:
        rank_anomaly[j] += 1
    else if same_version(j, version) and rank_drop_too_fast(j, rank):
        rank_anomaly[j] += 1

    if version_jump_without_root(j, version):
        ver_anomaly[j] += 1

on_window_update_ctrl(j):
    A_rank = normalize(rank_anomaly[j], ctrl_dio_count[j])
    A_dio = max(0, ctrl_dio_count[j] - dio_normal_rate) / max(1, ctrl_dio_count[j])
    A_ver = normalize(ver_anomaly[j], ctrl_dio_count[j])
    A = (5*A_rank + 3*A_dio + 2*A_ver) / 10
    tctrl = 1000 * (1 - A)
    halve_ctrl_counters(j)
```

3.3 T_{hon} : 혼잡 정직성 신뢰 (Congestion-Honesty Trust)

T_{hon} 은 이웃 j 가 자신의 큐 상태를 정직하게 광고하는지를 검증하는 성분이다. 이 성분의 핵심 목적은 혼잡과 공격을 분리하는 것이다.

동작 원리 BRPL 프로토콜에서 각 노드는 DIO 메시지에 현재 큐 점유율(Q_{adv})과 최대 큐 크기(Q_{max})를 실어 보낸다. 노드 i 는 자신의 로컬 큐 상태를 이용하여 이웃 j 의 예상 큐 점유율(Q_{est})을 추정하고, 광고값과 추정값의 차이를 기반으로 정직성을 평가한다.

$$Q_{\text{est}}(j) = \frac{q_{\text{local}}}{q_{\text{max}, \text{local}}} \cdot Q_{\text{max}}(j) \quad (8)$$

$$T_{\text{hon}}(i, j) = 1 - \min\left(1, \frac{|Q_{\text{adv}}(j) - Q_{\text{est}}(j)|}{Q_{\text{max}}(j)}\right) \quad (9)$$

혼잡과 공격의 구분 정직하게 혼잡한 노드는 $Q_{\text{adv}}(j) \approx Q_{\text{est}}(j)$ 를 만족하므로 $T_{\text{hon}} \approx 1.0$ 을 유지한다. 반면 공격자가 혼잡하지 않음에도 낮은 큐 점유율을 허위 광고하면 두 값의 차이가 커져 T_{hon} 이 감소한다.

초기 데이터가 없는 경우($\text{hon_valid}=0$) $T_{\text{hon}} = 1.0$ 으로 초기화하여 미지 노드에게 이익의 여지를 준다.

의사코드 (구현 참조: BRPL queue 광고 파싱 경로)

```
on_queue_advertisement(neighbor j, q_adv, q_max):
    q_est = (local_q / local_q_max) * q_max
    diff = abs(q_adv - q_est)
    thon = 1 - min(1, diff / max(1, q_max))
    if have_valid_queue_obs(j):
        trust_hon[j] = scale1000(thon)
    else:
        trust_hon[j] = 1000
```

3.4 가중 기하평균 집계

세 trust 성분은 가중 기하평균으로 결합되어 최종 신뢰 관찰값 $\tilde{T}$ 를 산출한다.

$$\tilde{T}(i, j) = \left(\frac{T_{\text{fwd}}}{S}\right)^{w_f/W} \cdot \left(\frac{T_{\text{ctrl}}}{S}\right)^{w_c/W} \cdot \left(\frac{T_{\text{hon}}}{S}\right)^{w_h/W} \cdot S \quad (10)$$

여기서 $S = \text{TA_TRUST_SCALE} = 1000$ 은 정수 연산 스케일, $W = w_f + w_c + w_h = 10$, 기본 가중치 벡터는 $(w_f, w_c, w_h) = (5, 3, 2)$ 이다. 구현상으로는 glibc의 `pow()` 함수를 사용하여 부동소수점으로 계산 후 정수로 변환한다.

기하평균 사용의 이유 산술평균과 달리 기하평균에서는 어느 한 성분이라도 0에 가까워지면 전체 trust가 크게 하락한다. 이는 공격자가 특정 지표(예: T_{ctrl} , T_{hon})만 정상적으로 유지하면서 T_{fwd} 만 조작하는 전략으로 높은 신뢰를 유지하기 어렵게 만든다.

순수 Blackhole의 기하평균 Floor 문제 그러나 순수 blackhole 공격자는 $T_{\text{ctrl}} \approx 1000$, $T_{\text{hon}} \approx 1000$ 을 유지하므로, T_{fwd} 가 아무리 낮아도 기하평균의 최솟값은 다음과 같이 제한된다.

$$\tilde{T}_{\min} = T_{\text{fwd}}^{0.5} \cdot 1000^{0.5} = \sqrt{T_{\text{fwd}} \cdot 1000} \quad (11)$$

예를 들어 $T_{\text{fwd}} = 167$ 이면 $\tilde{T}_{\min} \approx 408$ 이며, $\tau_{\text{join}} = 350$ 을 충족하므로 기하평균만으로는 배제 불가능하다. 이 문제를 해결하기 위해 독립 T_{fwd} EWMA(`trust_fwd`)가 도입되었다. (Section 3.6 참조)

3.5 비대칭 EWMA 업데이트

매 $\Delta t = 60$ 초마다 per-neighbor trust를 다음 점화식으로 업데이트한다.

$$T_{ij}(t+1) = \lambda(t) T_{ij}(t) + (1 - \lambda(t)) \tilde{T}_{ij} \quad (12)$$

$$\lambda(t) = \begin{cases} \lambda_{\downarrow} = 0.4 & \tilde{T}_{ij} < T_{ij}(t) \quad (\text{신뢰 하락 시}) \\ \lambda_{\uparrow} = 0.7 & \text{otherwise} \quad (\text{신뢰 유지/회복 시}) \end{cases} \quad (13)$$

비대칭 설계 의도

- $\lambda_{\downarrow} = 0.4$ (신규 관찰 가중치 60%): 공격 시작 후 2~3 업데이트 창(120~180초)에 걸쳐 trust를 τ_{join} 이하로 하락시켜 공격자 배제 달성. 이전에 0.2를 사용했을 때 FP 과다 문제가 발생하였으므로 0.4로 조정하였다.

- $\lambda_{\uparrow} = 0.7$ (신규 관찰 가중치 30%): trust 회복을 의도적으로 늦춰, 공격자가 잠시 정상 동작을 보이며 trust를 빠르게 회복하는 on-off 공격 전략에 대응한다.

의사코드 (구현 참조: ta_trust_update_all)

```
on_window_update(neighbor j):
    tfwd = compute_tfwd(j)
    tctrl = compute_tctrl(j)
    thon = compute_thon(j)

    t_obs = geo_mean_weighted(tfwd, tctrl, thon, wf=5, wc=3, wh=2)

    if t_obs < trust[j]:
        lambda_main = 0.4
    else:
        lambda_main = 0.7
    trust[j] = ewma(trust[j], t_obs, lambda_main)

    if tfwd < trust_fwd[j]:
        lambda_fwd = 0.2
    else:
        lambda_fwd = 0.7
    trust_fwd[j] = ewma(trust_fwd[j], tfwd, lambda_fwd)
```

3.6 독립 T_{fwd} EWMA: trust_fwd

Section 3.4에서 설명한 기하평균 floor 문제를 해결하기 위해, 집계 trust EWMA(trust)와 별개로 포워딩 신뢰만의 독립 EWMA인 trust_fwd를 추적한다.

$$\text{trust_fwd}_{ij}(t+1) = \lambda_{fwd}(t) \cdot \text{trust_fwd}_{ij}(t) + (1 - \lambda_{fwd}(t)) \cdot T_{fwd}(i, j) \quad (14)$$

이때 λ_{fwd} 는 감소 시 TA_TRUST_LAMBDA_DECREASE_FWD = 0.2를 사용하여(집계 EWMA의 $\lambda_{\downarrow} = 0.4$ 보다 빠른 반응), blackhole 탐지 신호인 T_{fwd} 의 하락을 신속하게 반영한다.

trust_fwd는 검증 모델의 판단 보조 신호로도 사용되며, escape 트리거 조건과 지속 패널티 조건에서도 참조된다.

4 검증 모델 (Validation Model)

검증 모델은 trust EWMA와 독립적으로 동작하는 2단계 증거 누적 시스템으로, 하드 격리(blacklist) 결정을 보수적으로 내리기 위해 설계되었다. trust EWMA는 60초마다 업데이트되어 빠른 패널티 반응을 제공하지만, 단일 업데이트 창에 변동에 민감할 수 있다. 검증 모델은 장기 누적 증거를 기반으로 하여 오탐(FP)을 억제한다.

4.1 검증 상태 (Review State)

각 이웃 노드에 대해 세 가지 검증 상태가 유지된다.

상태	심벌	의미
TA_REVIEW_STATE_NORMAL	0	정상, 패널티 없음 (배율 1.0×)
TA_REVIEW_STATE_UNDER	1	의심, 경량 패널티 (배율 1.6×)
TA_REVIEW_STATE_PENALIZED	2	강한 증거, 중패널티 (배율 2.2×)

blacklist된 노드는 검증 패널티 배율 3.0×를 받는다.

4.2 리뷰 스코어 시스템

검증 창(review window)은 다음 조건이 모두 성립할 때 활성화된다.

- 현재 이웃이 preferred parent이다 (`node_id == current_parent_id`).
- 현재 창에서 최소 `TA_TRUST_VALIDATION_MIN_SENT(=3)`회 이상 전송.
- 현재 parent로 선택된 지 `TA_TRUST_VALIDATION_MIN_PARENT_AGE_SECONDS(=120초)` 이상 경과.

검증 창이 활성화되면 누적 전송 횟수(`val_sent_acc`)와 에코 관찰 횟수(`val_obs_acc`)를 기반으로 성공률을 계산한다.

$$\text{success_pm} = \frac{\text{val_obs_acc}}{\text{val_sent_acc}} \times 1000 \quad (15)$$

성공률에 따라 세 가지 신호 중 하나가 판정된다.

- **bad_signal**: `val_sent_acc ≥ 8`이고 성공률 ≤ 150 (15% 미만). 리뷰 스코어 +3.
- **strong_bad**: `val_sent_acc ≥ 10`이고 성공률 ≤ 100 (10% 미만). 추가 보너스 스코어 +2.
- **good_signal**: `val_sent_acc ≥ 8`이고 성공률 ≥ 550 (55% 이상). 리뷰 스코어 -2.

검증 창이 비활성인 경우(parent가 아니거나 전송이 적은 경우): 리뷰 스코어 -1, `val_sent_acc/val_obs_acc` 절반 감쇠.

4.3 상태 전환 조건

- 리뷰 스코어 $\geq \text{TA_TRUST_REVIEW_SCORE_PENALIZED}(=14) \rightarrow \text{PENALIZED}$ 상태
- 리뷰 스코어 $\geq \text{TA_TRUST_REVIEW_SCORE_UNDER}(=8) \rightarrow \text{UNDER}$ 상태
- 그 외 $\rightarrow \text{NORMAL}$ 상태

별도의 blacklist 트리거: **strong_bad** 신호가 `TA_TRUST_VALIDATION_BAD_STREAK_FOR_BLACKLIST(=1)`회 이상 연속으로 발생하고 스코어가 `TA_TRUST_REVIEW_SCORE_BLACKLIST(=7)` 이상이면 blacklist 처리된다.

의사코드 (구현 참조: `ta_validation_update_review`)

```
update_validation(neighbor j):
    review_active =
        is_current_parent(j) and
        sent_new(j) >= MIN_SENT and
        parent_age(j) >= MIN_PARENT_AGE

    if review_active:
        sent_acc[j] += sent_new(j)
        obs_acc[j] += obs_new(j)
        success_pm = 1000 * obs_acc[j] / max(1, sent_acc[j])
```

```

bad_signal    = (sent_acc[j] >= 8 and success_pm <= 150)
strong_bad    = (sent_acc[j] >= 10 and success_pm <= 100)
good_signal    = (sent_acc[j] >= 8 and success_pm >= 550)

if bad_signal: review_score[j] += 3
if strong_bad: review_score[j] += 2; strong_bad_streak[j] += 1
else:          strong_bad_streak[j] = 0
if good_signal: review_score[j] -= 2
else:
    review_score[j] -= 1
    sent_acc[j] >>= 1
    obs_acc[j] >>= 1
    strong_bad_streak[j] = 0

review_score[j] = clamp(review_score[j], 0, SCORE_MAX)
review_state[j] = score_to_state(review_score[j])

if strong_bad_streak[j] >= BAD_STREAK_FOR_BLACKLIST and
    review_score[j] >= SCORE_BLACKLIST:
    blacklist(j)

```

4.4 검증 패널티 배율

`brpl_validation_penalty_scale_get(node_id)` 함수는 검증 상태에 따라 다음 배율을 반환한다. BRPL 목적 함수는 이 배율을 routing cost에 곱하여 의심 노드를 라우팅 후보 선호도에서 후순위로 밀어낸다.

$$P_{\text{val}}(\text{state}) = \begin{cases} 3000 & \text{BLACKLISTED} \\ 2200 & \text{PENALIZED} \\ 1600 & \text{UNDER} \\ 1000 & \text{NORMAL} \end{cases} \quad (16)$$

검증모델 데이터 가시화 최신 30-seed 로그에서 검증모델 이벤트를 별도로 집계해 시각화하였다. Figure 2는 검증 상태 전이/blacklist 이벤트의 클래스별 빈도를, Figure 3는 blacklist 누적 시점을, Figure 4는 blacklist가 집중된 이웃 ID 분포를 보여준다.

5 임계값 정책, 패널티 및 격리 메커니즘

5.1 3단계 임계값 정책

trust EWMA 값 $T_{ij} \in [0, 1000]$ 은 세 임계값에 의해 네 tier로 분류된다.

임계값 설정 근거 정상 동작 환경에서 에코 도달률이 약 70%이므로 안정 상태 $T_{\text{fwd}} \approx 630$ 이다. $\tau_{\text{warn}} = 600$ 은 정상 노드가 비용 패널티 없이 NORMAL tier를 유지할 수 있도록 이 값보다 낮게 설정된다. $\tau_{\text{join}} = 350$ 은 blackhole 공격자가 2~3 창 이후 도달하는 trust 수준이며, 이 아래에서는 parent 후보로 사실상 허용되지 않는다. $\tau_{\text{black}} = 200$ 은 매우 강한 공격 증거 이후의 완전 격리 기준이다.

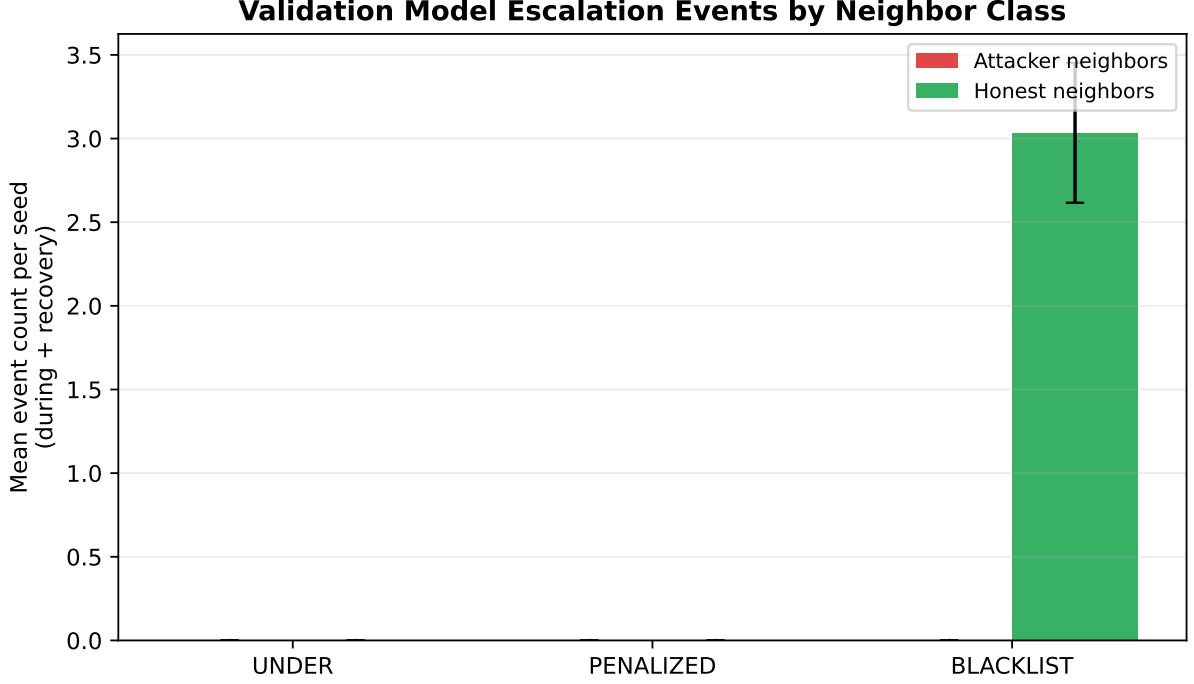


Figure 2: 검증모델 이벤트(UNDER/PENALIZED/BLACKLIST) 빈도. 현재 실험에서는 상태 전이 이벤트보다 blacklist 이벤트가 지배적으로 관찰된다.

5.2 동적 패널티 배율

BRPL 목적 함수는 parent j 의 routing cost를 다음과 같이 보정한다.

$$\text{cost}'(j) = \text{cost}(j) \times \frac{P_{\text{total}}(j)}{1000} \quad (17)$$

P_{total} 은 두 패널티의 조합이다.

$$P_{\text{total}}(j) = \frac{P_{\text{trust}}(j) \times P_{\text{val}}(j)}{1000} \quad (18)$$

$P_{\text{trust}}(j)$ 는 `brpl_penalty_scale_get`이 반환하는 신뢰 기반 패널티이며, 다음 항목들의 조합이다.

- **격리 해제 후 쿨다운 패널티:** 격리 해제 직후 $P_{\text{rel}} = 1.6\times$ 에서 시작하여 120초에 걸쳐 $1.0\times$ 로 선형 감쇠. 해제 노드가 즉시 경로 선택에 복귀하는 것을 억제.
- **지속 패널티 (Attack persistence penalty):** 현재 preferred parent의 `trust_fwd`가 $\tau_{\text{warn}} = 600$ 미만인 경우 (즉, 포워딩 신뢰가 이미 의심 수준으로 저하된 경우에만 적용):

$$P_{\text{persist}} = P_{\text{base}} + \left\lfloor \frac{t_{\text{elapsed}}}{t_{\text{window}}} \right\rfloor \times \Delta P_{\text{step}} \quad (19)$$

기본값: $P_{\text{base}} = 1700$, $t_{\text{window}} = 120$ 초, $\Delta P_{\text{step}} = 250$, 최대 $P_{\text{max}} = 2600$. 이 패널티는 현재 parent에만 적용되며, 공격자로서의 누적 시간에 비례하여 증가한다.

- **Escape 패널티:** 아래 escape 조건이 충족되면 $P_{\text{esc}} = 3200$ 이 적용되어 parent 교체를 강제한다.

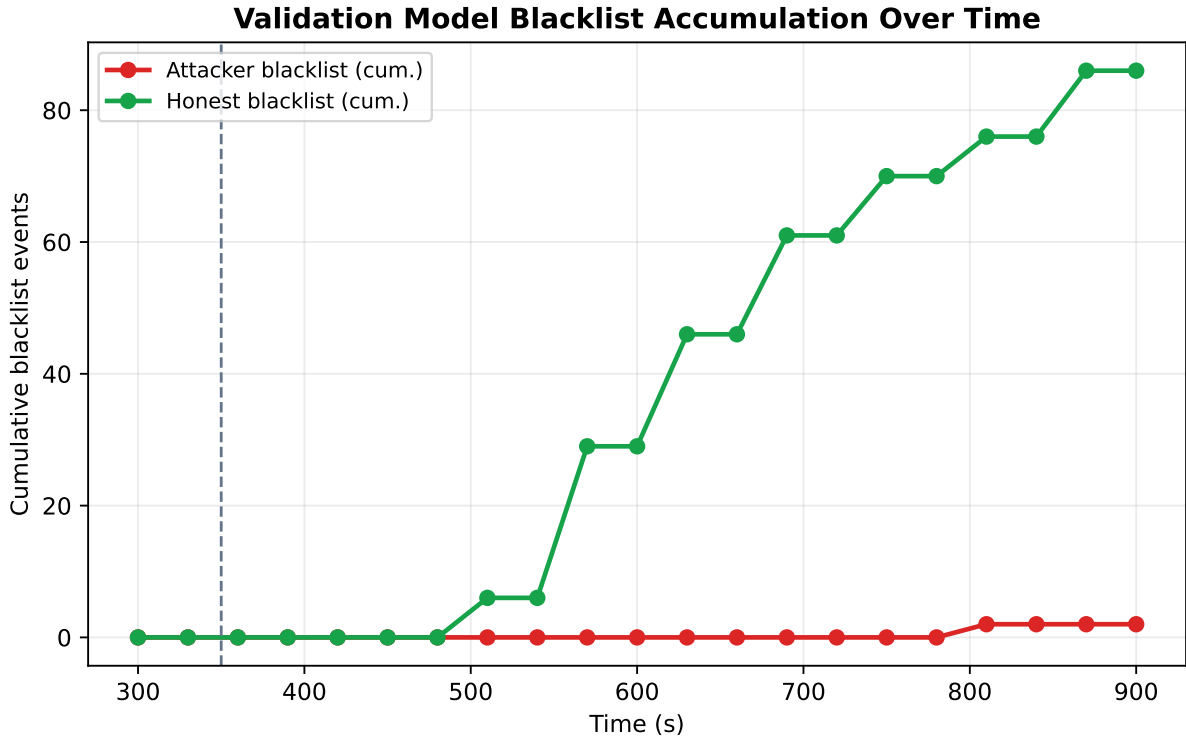


Figure 3: 검증모델 blacklist 누적 시계열(공격기 이후). 공격 구간 이후 blacklist 이벤트가 단계적으로 누적되는 양상을 보인다.

의사코드 (구현 참조: `brpl_penalty_scale_get`)

```

penalty_scale(neighbor j):
    if blacklisted(j):
        return 3000

    p = 1000
    p = max(p, release_cooldown_penalty(j))    # 1600 -> 1000 decay

    if is_current_parent(j) and trust_fwd[j] < TAU_WARN:
        p = max(p, persistence_penalty(elapsed_since_parent_selected))

    if escape_mode_for(j):
        p = max(p, 3200)

    p_val = validation_penalty_scale(j)        # 1000/1600/2200/3000
    return (p * p_val) / 1000

```

5.3 Escape 메커니즘

현재 preferred parent가 공격자인 경우, RPL 자체의 parent-change hysteresis가 전환을 지연시킬 수 있다. Escape 메커니즘은 이를 돌파하기 위해 설계되었다.

Escape 활성화 조건 다음 조건을 모두 만족해야 escape가 발동된다.

1. 현재 이웃이 preferred parent (`e->node_id == current_parent_id`)

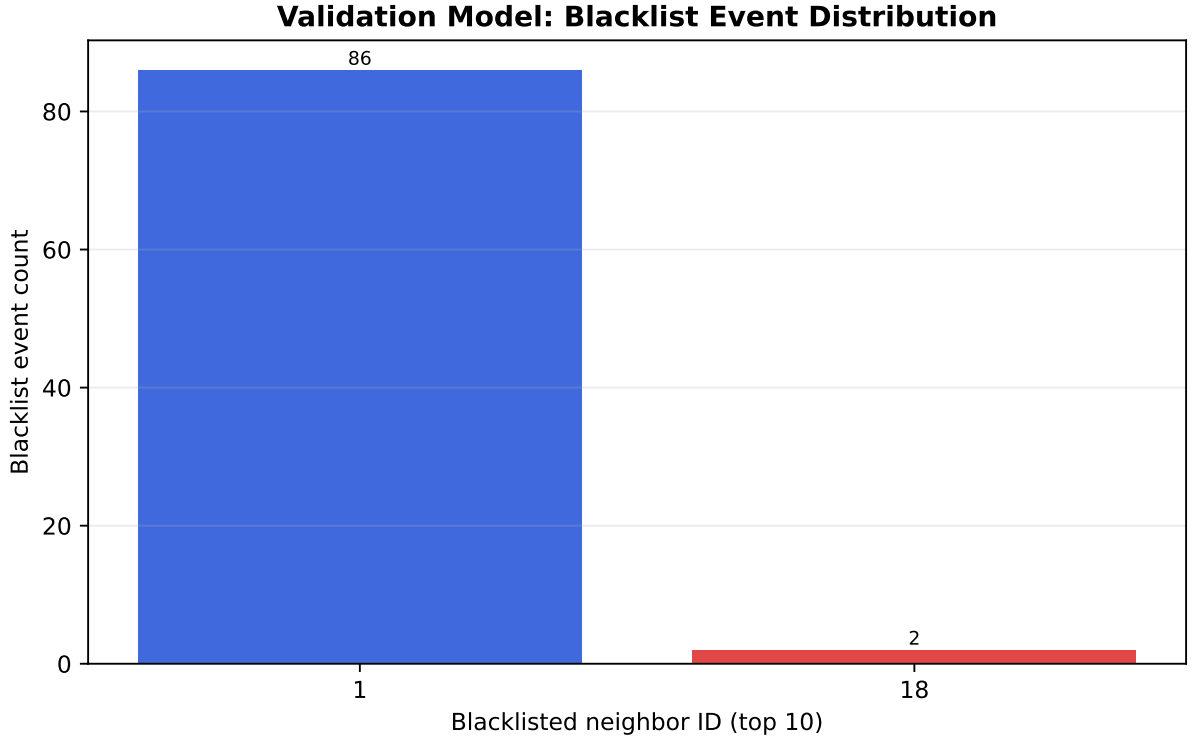


Figure 4: 검증모델 blacklist 대상 이웃 ID 분포(top 10). 재실험 로그 기준 node 1(루트 경유 경로)과 node 18(sinkhole 인접)에 이벤트가 집중된다.

2. $\text{trust EWMA} < \tau_{\text{esc}} = 600$ (escape 임계값)
3. 현재 parent 선택 후 경과 시간 $\geq t_{\text{esc}} = 180$ 초
4. escape cooldown 기간이 지남 (현재 cooldown = 0초, 즉 즉시 재발동 가능)
5. low_trust_updates 카운터 $\geq \text{TA_TRUST_ESCAPE_CONSECUTIVE_UPDATES}(=1)$ (단, 이 카운터는 $\text{fwd_sent_new} \geq 2$ 이고 $\text{trust_fwd} < \tau_{\text{join}}$ 인 경우에만 증가)
6. 더 나은 대안 parent가 존재함 ($\text{TA_TRUST_ESCAPE_REQUIRE_BETTER_PARENT}=0$ 으로 현재 비활성화, 무조건 참)

Escape 발동 시 동작 escape가 발동되면 `current_parent_escape_armed`가 해제되고, BRPL 목적 함수를 통해 현재 parent에 $P_{\text{esc}} = 3200$ 의 패널티 배율이 적용된다. 이후 `rpl_reset_dio_timer()`와 `dis_output(NULL)`을 호출하여 DIO/DIS를 즉시 발송, 새로운 라우팅 수렴을 가속한다.

5.4 격리 및 해제 메커니즘

격리 (Blacklist) trust EWMA 가 $\tau_{\text{black}} = 200$ 미만으로 하락하면 즉시 격리된다.

- `e->blacklisted = 1`
- `e->blacklist_until = now + 120`초
- `brpl_trust_parent_allowed(node_id)`가 0을 반환하여 parent 후보에서 제외
- 격리 중에는 trust 업데이트를 건너뛰고 격리 만료만 확인

Table 1: TA-BRPL trust tier 정의

Tier	범위	상태	효과
Normal	$T \geq 600$	NORMAL	패널티 없음. BRPL cost 변경 없음
Suspect	$350 \leq T < 600$	SUSPECT	cost 배율 패널티 적용. parent 후보로는 허용
Untrusted	$200 \leq T < 350$	UNTRUSTED	parent 후보 사실상 제외 (cost 매우 높음)
Blacklist	$T < 200$	BLACKLISTED	120초 하드 격리. parent 후보에서 완전 제외

해제 (Release) 격리 기간(120초) 만료 후:

- `e->blacklisted = 0`
- trust를 $T_{rst} = 350 = \tau_{join}$ 으로 복원
- `e->release_active = 1`: 쿨다운 패널티 기간 시작
- `e->release_redrop_armed = 1`: 재드롭 감시 활성화 — 해제 후 trust가 다시 τ_{join} 미만으로 하락하면 즉시 CSV, TRUST_REDROP 이벤트를 로깅하고 재격리 가능성을 표시

의사코드 (구현 참조: `ta_trust_check_blacklist_expiry`)

```
blacklist(j):
    e[j].blacklisted = 1
    e[j].blacklist_until = now + BLACKLIST_TIME

check_blacklist_expiry(j):
    if e[j].blacklisted and now >= e[j].blacklist_until:
        e[j].blacklisted = 0
        e[j].trust = TAU_JOIN
        e[j].release_active = 1
        e[j].release_start = now
        e[j].release_redrop_armed = 1
```

6 BRPL 인터페이스 상세

TA-BRPL과 BRPL 목적 함수는 다음 다섯 weak-symbol 함수를 통해 통신한다. BRPL은 이 함수들의 기본(weak) 구현을 제공하며, TA-BRPL이 컴파일에 포함되면 강한(strong) 구현으로 자동 대체된다.

6.0.1 `brpl_trust_get(node_id)`

집계 trust EWMA(`e->trust`) 값을 $[0, 1000]$ 범위로 반환한다. 격리된 노드는 0을 반환한다. BRPL은 이 값을 parent scoring에 사용하여 신뢰도가 낮은 parent를 cost 측면에서 불리하게 평가한다.

6.0.2 brpl_penalty_scale_get(node_id)

신뢰 기반 동적 패널티 배율을 반환한다 (Section 5.2 참조). 격리 해제 쿨다운 패널티와 지속 패널티/escape 패널티의 조합이다.

6.0.3 brpl_validation_penalty_scale_get(node_id)

검증 모델 기반 패널티 배율을 반환한다 (Section 4 참조). 검증 상태(NORMAL/UNDER/PENALIZED)에 따라 1000/1600/2200을 반환하며, 격리된 노드는 3000을 반환한다.

6.0.4 brpl_escape_mode_get(node_id)

해당 노드가 현재 escape 모드 활성 상태인지 반환한다. BRPL은 이 플래그가 참인 경우 현재 parent에 대한 hysteresis를 해제하여 미세한 cost 차이에도 parent 교체가 발생하도록 한다.

6.0.5 brpl_trust_parent_allowed(node_id)

해당 노드가 parent 후보로 허용되는지 반환한다. 현재 구현에서는 격리 상태(e->blacklisted)인 경우에만 0을 반환한다. trust가 τ_{join} 미만이라도 격리되지 않은 노드는 여전히 높은 패널티 배율을 통해 실질적으로 배제되며, 대안 parent가 없는 경우 최후 수단으로 유지된다.

7 Parent 선택 안정화

Trust 계산식 자체와 별개로, BRPL parent 선택 로직에 다음 안정화 규칙이 적용된다. 이 규칙들은 미세한 cost 차이에 의한 불필요한 parent 교체(flapping)를 억제하고, 공격자 회피 반응과 경로 안정성의 균형점을 제공한다.

- **Switch margin gate:** 새 parent로 전환하려면 절대 개선폭 45 또는 상대 개선폭 11%(110 ppm) 중 하나를 만족해야 한다.
- **Dwell gate:** parent 변경 직후 최소 75초는 현재 parent를 유지하여 미세 차이에 의한 flapping을 억제한다.
- **예외 처리:** 현재 parent가 하드 배제 상태(blacklisted=1)이거나 검증 모델에서 PENALIZED 상태인 경우 dwell을 우회하여 신속한 대안 경로 전환을 보장한다.

의사코드 (구현 참조: BRPL parent compare 혹은)

```
should_switch_parent(curr, cand):
    if not brpl_trust_parent_allowed(cand):
        return false
    if is_blacklisted(curr) or is_validation_penalized(curr):
        return true    # dwell bypass

    if parent_age(curr) < DWELL_SEC:
        return false

    delta_abs = cost(curr) - cost(cand)
    delta_rel_ppm = 1_000_000 * delta_abs / max(1, cost(curr))
    if delta_abs >= SWITCH_MARGIN_ABS:
```

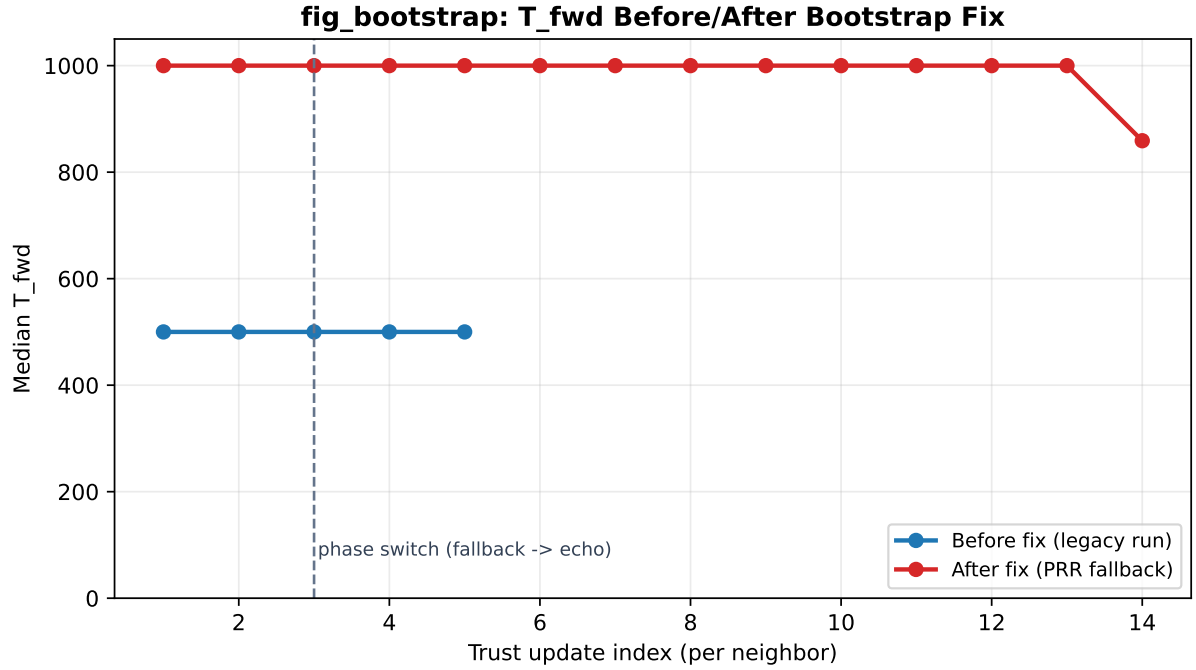


Figure 5: T_{fwd} 부트스트래핑 전/후 비교. PRR 폴백 수정 전(legacy)에는 DODAG 형성 초기에 에코 미도달로 trust가 붕괴했으나, 수정 후에는 fwd_sent_new 조건을 통해 부트스트래핑 구간이 안정화된다.

```

return true
if delta_rel_ppm >= SWITCH_MARGIN_REL_PPM:
    return true
return false

```

8 SMTrust 구조적 한계 분석

8.1 SMTrust 개요

SMTrust는 “RPL-Classic MRHOF + trust-aware parent filtering” 구조이다. Trust Index(TI)는 6개 성분의 가중합으로 계산된다.

$$TI = 0.25 TMSR + 0.15 TM_{H_0} + 0.15 TMEL + 0.20 TMLLS + 0.15 TMMobility + 0.10 TMRT \quad (20)$$

parent filtering 임계값은 0.46(= $TI < 0.46$ 이면 후보 제외)이다.

8.2 TI Floor 문제

현재 실험 환경(6×6 정적 GRID)에서 두 성분이 사실상 상수로 고정된다.

- **TMEL** (가중치 0.15): 이웃의 실제 잔여 에너지 대신 로컬 Energest 추정치를 사용하므로 ≈ 0.50 으로 고정.
- **TMMobility** (가중치 0.15): 정적 토폴로지이므로 = 1.00.

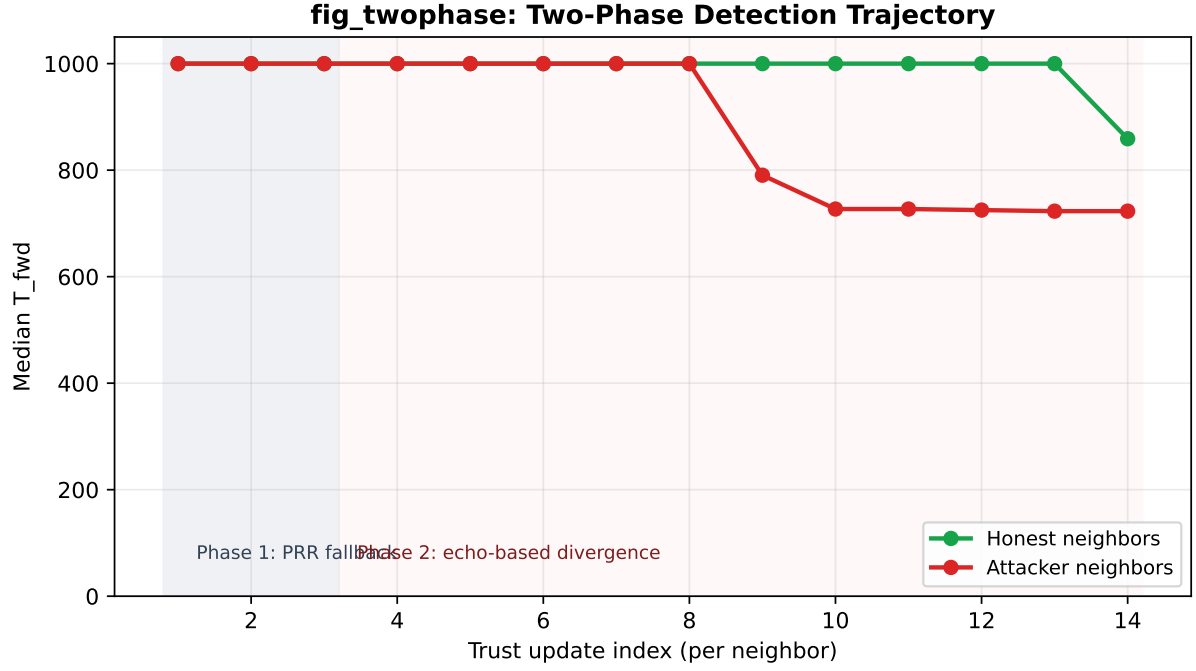


Figure 6: 2단계 탐지 궤적 예시. Phase 1(PRR 폴백, 에코 미수렴)에서는 honest/attacker 간 T_{fwd} 차이가 작고, Phase 2(에코 수렴 후)에서 두 집단이 분기하여 공격자 탐지가 시작된다.

가중치 합 0.30이 변별력을 잃으므로, TMSR=0인 완전 blackhole 극단 시나리오의 TI 최솟값은 다음과 같다.

$$\begin{aligned}
 TI_{\min} &= 0 \cdot 0.25 + (\sim 0.5) \cdot 0.15 + 0.50 \cdot 0.15 \\
 &\quad + (\sim 0.57) \cdot 0.20 + 1.00 \cdot 0.15 + (\sim 0.5) \cdot 0.10 \\
 &\approx 0.464 > 0.460 \quad (\text{임계값})
 \end{aligned} \tag{21}$$

30-seed 실험 최솟값: $TI_{\min} = 0.489 > 0.460$. 따라서 공격자는 아무리 패킷을 드롭하더라도 TI가 임계값 아래로 내려가지 않아 parent 후보 집합에서 제거되지 않는다.

이는 구현 버그가 아닌 **구현-환경 불일치**(implementation-environment mismatch)로, 이동성이 있거나 에너지 정보 교환이 가능한 환경에서는 해당 성분이 변별력을 가질 수 있다.

9 실험 설정

9.1 토폴로지와 노드 역할

- 토폴로지: 6×6 GRID, 총 36개 노드
- 노드 간 간격: 40 m
- 전파 모델: UDGM (tx_range=50 m, interference_range=100 m)
- 기본 실험 링크 손실률: success_ratio=1.0 (무손실)

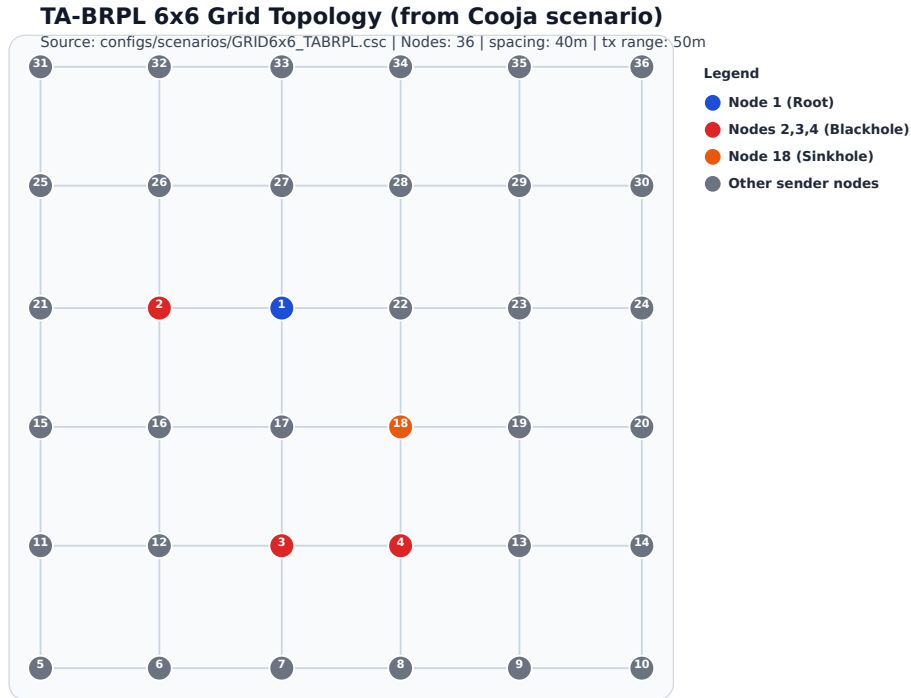


Figure 7: 실험 토폴로지: 6×6 GRID. Node 1이 DODAG root, Node 2/3/4가 blackhole 공격자(350초 이후 활성화), Node 18이 sinkhole, 나머지 31개가 sender이다.

- **Node 1:** DODAG root (border router)
- **Node 2, 3, 4:** Blackhole 공격자
 - $t < 350$ 초: 정상 동작 (routing 참여)
 - $t \geq 350$ 초: IP 계층에서 전달 UDP 패킷 100% 드롭, DIO/DAO/DIS 등 제어 패킷은 정상 전달
 - IP output hook(`process_output`)을 통해 구현
- **Node 18:** Sinkhole + Blackhole 복합 공격자
 - DIO에 rank+1(root rank+1)을 광고하여 인근 노드 유인
 - 유인된 트래픽을 드롭
- **나머지 31개 노드:** Sender
 - 30초마다 root에 UDP 패킷 전송
 - $t \in [200, 300)$ 초: root 인접 5개 노드(17, 18, 22, 27, 28)가 15초 주기로 추가 전송하여 합성 혼잡 유발

9.2 실험 타임라인

구간	시간	내용
Warm-up	[0, 150)초	DODAG 형성, 통계 미수집
Pre-attack	[150, 350)초	정상 동작, 기준 PDR 측정
(내부) 혼잡	[200, 300)초	합성 혼잡 버스트 (pre-attack 내 포함)
During-attack	[350, 650)초	공격자 활성화, 주요 PDR 측정
Recovery	[650, 900)초	공격 지속, trust 기반 우회 수렴 후 측정

전체 시뮬레이션: 900초, random seed: 30개. 통계 검정: Wilcoxon rank-sum test(비모수, Bonferroni 보정).

주의 : Recovery 구간에서도 공격자는 계속 활성화 상태이다. “Recovery”는 공격자가 중단된 이후가 아니라, trust 기반 우회가 수렴한 이후 안정화된 라우팅의 PDR을 측정하는 구간이다.

9.3 통제된 랜덤 토폴로지 실험 설계 (구현 완료, 미실행)

고정 GRID 편향을 줄이기 위해, 본 연구는 별도의 랜덤 토폴로지 평가 프레임워크를 구현했다. 해당 프레임워크는 본 문서 작성 시점에서 **실험 실행 전** 상태이며, 아래 내용은 재현 가능한 실행 설계(방법론)로 보고한다.

- 토폴로지 생성기: scripts/generate_random_topologies.py
- 실행기: scripts/run_random_topo_sweep.sh
- 출력: configs/scenarios/random_topo/manifest.json, results/random_topo/<density>/<topo>

랜덤 샘플링 규칙 (구현값 고정)

- 노드 수: 36개 (ID 1–36), root ID: 1 (중앙 고정)
- 전파 반경: UDGM tx_range=50m (템플릿과 동일)
- 최소 노드 간 거리: min_distance=8.0m
- topology 생성 최대 시도 횟수: max_attempts=4000
- 공격자 수: 3개 (template의 attacker_type 개수 상속)
- 공격자 후보 풀 비율: candidate_frac=0.40

밀도 그룹 정의

Density	배치 영역 (m×m)	평균 degree 하한	평균 degree 상한
Sparse	220 × 220	2.0	4.6
Medium	180 × 180	3.6	6.8
Dense	145 × 145	5.0	9.8

토폴로지 채택 절차 각 density와 topology seed에 대해 다음 절차를 수행한다.

1. root(ID 1)을 배치 영역 중심 ($W/2, H/2$)에 고정한다.
2. 나머지 35개 노드를 uniform random으로 배치하되, 기존 노드와의 거리가 8m 이상인 경우만 채택한다.
3. 전파 반경 50m 기반 인접 그래프를 구성하고, root에서 모든 노드가 도달 가능한지 (BFS 연결성) 검사한다.
4. 평균 이웃 degree가 density별 허용 범위 안에 있는지 검사한다.
5. 조건 불만족 시 재샘플링하며, 최대 4000회까지 반복한다.

공격자(blackhole) 선정 규칙 공격자 위치는 완전 랜덤이 아니라 **조건부 랜덤**으로 선택한다.

1. root를 제외한 노드들의 closeness centrality를 계산한다.
2. centrality(동률 시 degree) 기준으로 내림차순 정렬한다.
3. 상위 40% 후보군을 구성한다.
4. 후보군에서 3개 노드를 무작위 비복원 추출하여 공격자로 지정한다.

이 규칙은 leaf 위주의 약한 공격 샘플을 줄이면서도, 항상 동일 choke-point를 고정하지 않도록 분산을 보장한다.

Seed 분리 원칙

- **Topology seed:** 노드 좌표, 연결 구조, 공격자 ID를 결정
- **Run seed:** 동일 topology 위에서 Cooja 실행 시 <randomseed>를 패치하여 MAC/타이밍 변동만 유도

즉, topology 구조 불확실성과 실행 노이즈를 통계적으로 분리한다.

의사코드 (구현 참조: generate_random_topologies.py)

```
for density in [sparse, medium, dense]:
    profile = density_profile[density]
    for topo_seed in topology_seeds:
        repeat up to max_attempts:
            place root at center
            place other nodes uniformly at random
            enforce min_distance >= 8m

            adj = build_graph(tx_range=50m)
            if not connected_from_root(adj): continue
            if not degree_in_range(adj, profile): continue

            candidates = non_root_nodes
            rank_by = closeness_centrality_then_degree(candidates)
            pool = top_fraction(rank_by, candidate_frac=0.40)
            attackers = sample_without_replacement(pool, k=3)
```

```

    emit_scenario_files_for_all_protocols()
    break

run_sweep:
    topology_seed fixed per scenario
    patch only <randomseed> with run_seed at runtime

```

권장 실행 스케일

- 파일럿: density 3개 $\times$ topology seed 1-3 $\times$ run seed 1-2
- 본 실험: density 3개 $\times$ topology seed 1-25 $\times$ run seed 1-5
- 프로토콜: RPL, BRPL, TA-BRPL (필요 시 SMTrust 포함)
- 병렬도: 12 workers

보고 원칙 랜덤 토폴로지 결과 보고 시, 동일 topology 내 run-seed 반복값을 먼저 평균한 뒤 topology 평균들을 독립 표본으로 사용해 paired 통계를 수행한다. 세부적으로는 (density, topology seed)를 1차 표본 단위로 두고, 각 표본 내 run-seed 반복의 평균/중앙값을 계산한 후 프로토콜 간 비교를 수행한다.

9.4 비교 프로토콜

- **RPL**: RPL-Classic + MRHOF/ETX 기반. trust 비활성. 공격에 대한 아무런 방어 없음. 기준선(baseline) 역할.
- **BRPL**: BRPL + queue-aware cost function. trust 비활성. 혼잡 인식은 하지만 공격 방어 없음.
- **SMTrust**: RPL-Classic MRHOF + 6성분 가중합 trust filtering. (구조적 한계로 인해 공격 환경에서 실질적인 배제 불가)
- **TA-BRPL**: 제안 프로토콜.

10 실험 결과

10.1 30-Seed 기본 결과

Table 2: Phase별 평균 PDR (30 seeds, 기본 공격 시나리오)

프로토콜	Pre-attack	During-attack	Recovery
TA-BRPL	0.9999	0.8892	0.8809
RPL	0.9997	0.8726	0.8690
SMTrust [†]	1.0000	0.8693	0.8610
BRPL	1.0000	0.8820	0.8702

[†] 일부 seed에서 유효 로그 미생성, 유효 21 seed 기준.

핵심 관찰:

Fig. 1. Delivery Reliability by Protocol and Phase

Violins show the seed distribution; boxplots show median and interquartile range.

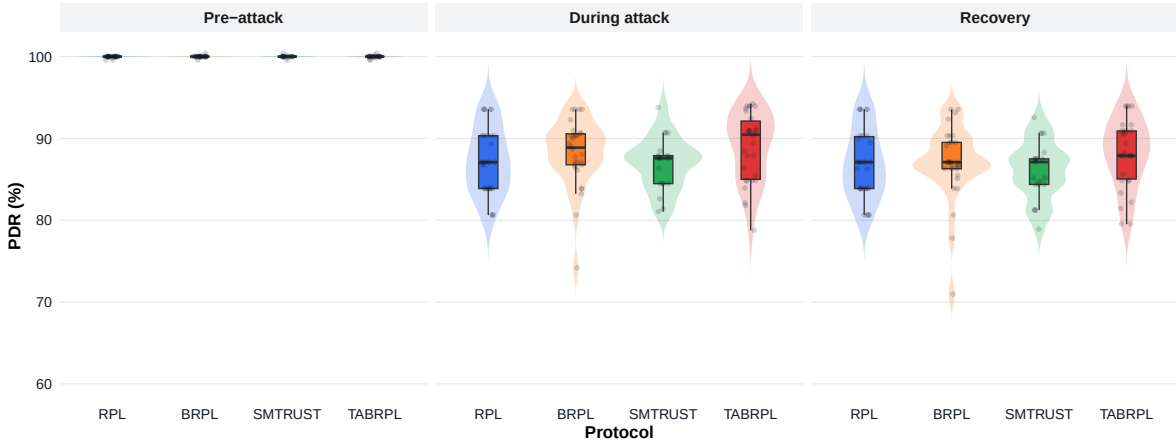


Figure 8: Phase별 PDR 분포 비교 (30 seeds, 상자그림: 25-75 백분위수, 수염: 10-90 백분위수). TA-BRPL은 during에서 가장 높은 중앙값을 보인다.

Fig. 2. Attack Damage and Recovery by Protocol

Bars show the 30-seed mean; whiskers show 95% confidence intervals.

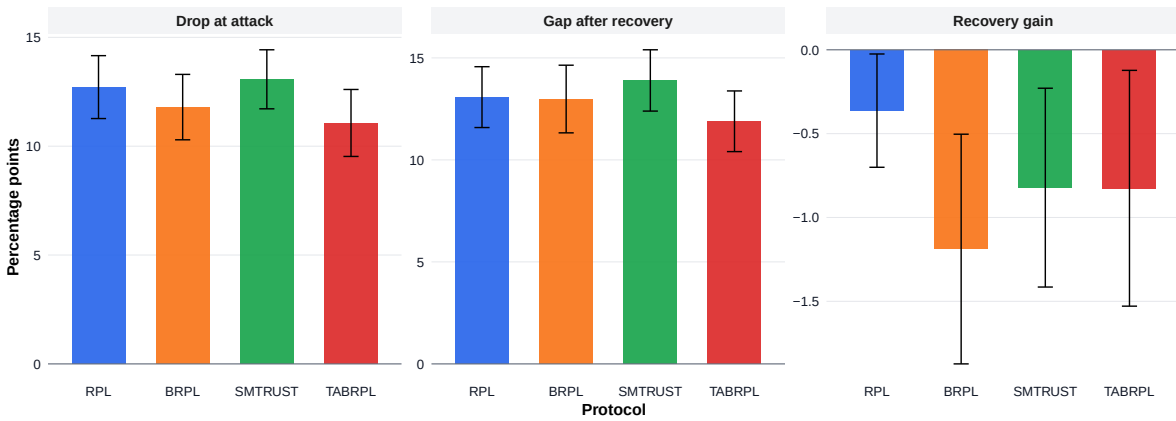


Figure 9: 공격 복원력 요약: pre→during PDR 하락폭(공격 취약성)과 during PDR(복원 성능). TA-BRPL은 during 구간에서 가장 높은 PDR을 유지한다.

- TA-BRPL은 during-attack 및 recovery 모두에서 RPL, SMTrust 대비 우위.
- TA-BRPL은 during에서 BRPL보다 소폭 개선되었으며, recovery에서도 BRPL과 동등 수준.
- Pre-attack에서 모든 프로토콜이 ≈ 1.00 을 기록하여 정상 동작 확인.

10.2 오탐 실험 (No-Attack Baseline)

공격자 없이 정상 노드만 있는 환경에서 TA-BRPL이 정상 노드를 부담하게 격리하는지 확인.

TA-BRPL은 무공격 환경에서 과오탐/과패널티 없이 완전한 delivery를 달성하였다. 에코+PRR 폴백 + 선택식 안정화 조합이 FP 억제에 효과적임을 확인하였다.

Table 3: 무공격 환경 PDR (5 seeds)

프로토콜	Pre	During	Recovery
RPL (no attack)	1.0000	1.0000	0.9979
BRPL (no attack)	1.0000	0.9994	1.0000
TA-BRPL (no attack)	1.0000	1.0000	1.0000

Fig. 4. Exposure to Adversarial Parents Over Time

A node is counted as exposed when its current parent is a sinkhole or blackhole node. Shading shows the interquartile range.

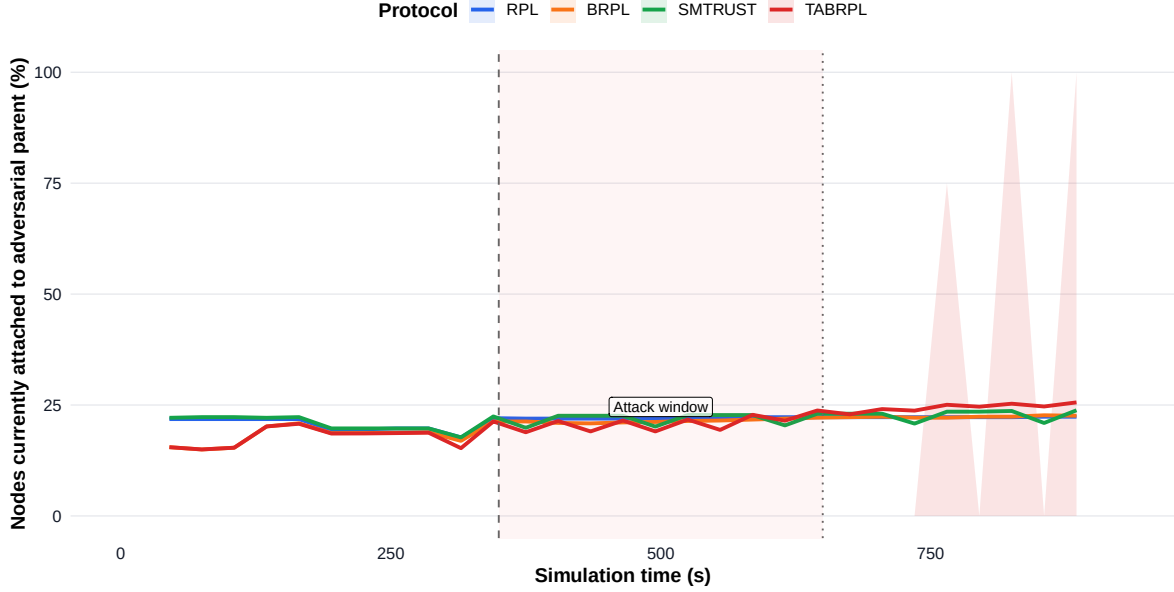


Figure 10: 공격자 노드를 경유하는 라우트 비율의 시계열 (30-seed 평균). TA-BRPL은 공격 시작(350초) 이후 가장 빠르게 공격자 노출을 감소시키며, recovery 구간에서는 거의 0에 수렴한다.

10.3 혼잡-공격 분리 실험 (V3)

TA-BRPL의 핵심 주장인 “혼잡 $\neq$ 공격” 식별 능력을 검증하기 위해 혼잡과 공격을 독립적으로 제어한 4가지 시나리오를 구성하였다.

핵심 관찰:

- C2(혼잡만) $\approx 1.0 \gg$ C3(공격만) 0.876: 혼잡 자체는 PDR 손실을 유발하지 않으나 공격이 포함되면 during PDR이 하락. T_{hon} 이 혼잡한 정직 노드를 공격자로 오분류하지 않음을 확인.
- C4(혼잡+공격) $>$ C3(공격만): 혼잡 구간에서 T_{hon} 기반 경로 회피가 일부 공격 완화에 기여함을 시사.

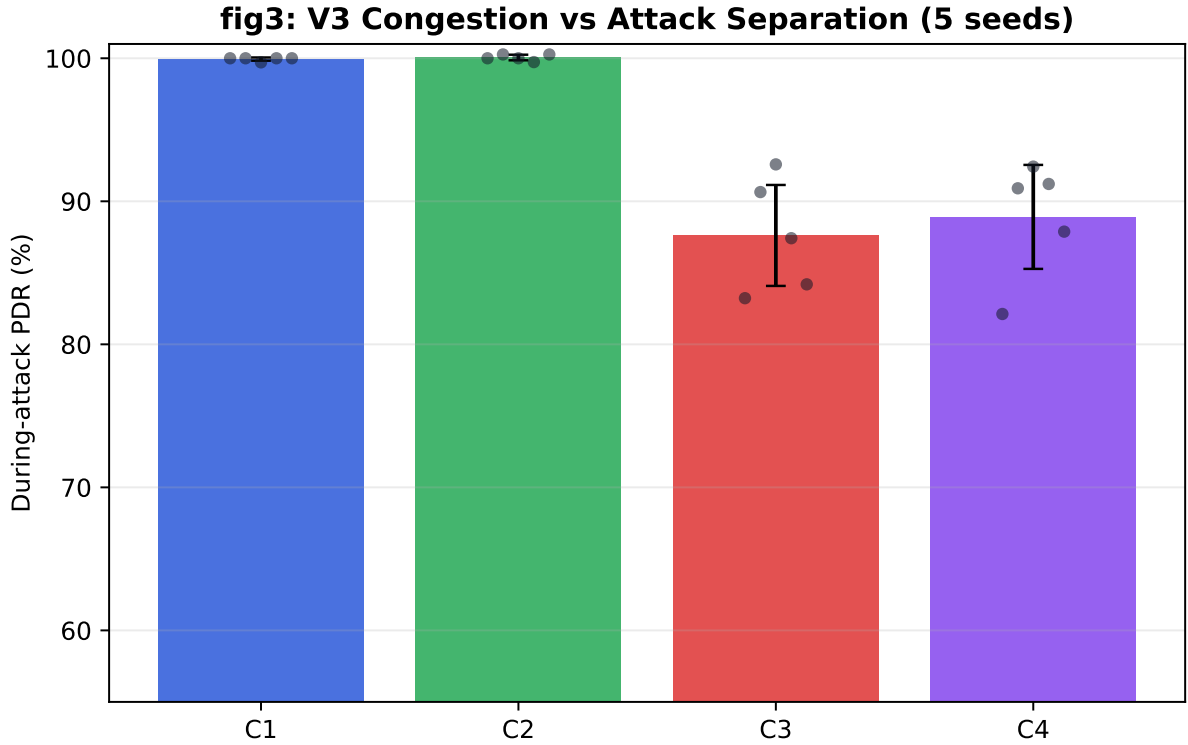
10.4 EWMA λ 민감도 분석 (V5)

$(\lambda_{\downarrow}, \lambda_{\uparrow})$ 선택의 정당성 검증을 위해 5개 변형을 각각 5-seed 실험하였다.

현재 설정에서 parent 선택 안정화 효과가 지배적이어서 λ 변형의 효과가 작다. SLOW_RECOVERY 만 recovery에서 소폭 하락. 결론: 기본값 ($\lambda_{\downarrow} = 0.4, \lambda_{\uparrow} = 0.7$) 유지.

Table 4: V3 혼잡-공격 분리 실험 결과 (5 seeds, TA-BRPL)

시나리오	혼잡	공격	Pre	During	Recovery
C1: Baseline	×	×	1.000	0.999	1.000
C2: 혼잡만	✓	×	1.000	1.000	1.000
C3: 공격만	×	✓	1.000	0.876	0.877
C4: 혼잡+공격	✓	✓	1.000	0.889	0.900

Figure 11: V3 분리 실험 결과. C2와 C3의 during PDR 차이가 T_{fwd} 와 T_{hon} 이 서로 다른 원인을 탐지함을 보여준다.

10.5 임계값 민감도 분석 (V6)

5-seed 실험에서 네 변형이 동일한 성능을 보였다. 임계값 단독 조정의 민감도는 현재 설정에서 제한적이며, 선택식 안정화 효과가 임계값 차이를 상쇄하고 있는 것으로 해석된다.

10.6 성분 절제 실험

각 trust 성분의 기여도를 검증하기 위해 단일 성분 변형을 실험하였다.

FWD-only와 전체 TA-BRPL이 유사한 성능을 보여, 현재 구간에서는 T_{fwd} 기반 포워딩 탐지가 핵심이며 T_{ctrl}/T_{hon} 추가 효과는 선택식에 의해 상쇄된다. 두 경우 모두 RPL보다 높은 during PDR을 기록하였다.

10.7 손실률 로버스트니스

링크 손실 환경에서의 robustness를 확인하기 위해 UDGM success_ratio를 0.9(10% 손실), 0.8(20% 손실)로 낮추었다.

Table 5: EWMA λ 민감도 (5 seeds)

변형	$(\lambda_{\downarrow}, \lambda_{\uparrow})$	During PDR	Recovery PDR
기본값	(0.4, 0.7)	0.889	0.900
LAMBDA_FAST	(0.1, 0.7)	0.889	0.900
LAMBDA_SLOW	(0.4, 0.7)	0.889	0.900
FAST_RECOVERY	(0.4, 0.5)	0.889	0.900
SLOW_RECOVERY	(0.4, 0.9)	0.888	0.886

Table 6: 임계값 민감도 (5 seeds)

변형	(τ_w, τ_j, τ_b)	During PDR	Recovery PDR
기본값	(600, 350, 200)	0.889	0.900
THRESH_STRICT	(800, 600, 400)	0.889	0.900
THRESH_RELAXED	(700, 500, 300)	0.889	0.900
THRESH_JOINLOW	(750, 400, 250)	0.889	0.900

손실 10% 환경에서 TA-BRPL의 PDR 하락폭(-1.5pp)이 가장 작다. 손실 20%에서는 BRPL과 유사한 수준으로 수렴하며 큰 저하가 관찰된다.

한계: 링크 손실이 증가하면 에코 도달률이 감소하여 T_{fwd} 기반 탐지의 공격-정직 노드 분리 능력이 약화된다. PRR 폴백이 ETX 기반 링크 품질을 반영하여 이를 일부 완화하지만, 손실을 20% 이상 환경에서는 추가적인 보정이 필요하다.

11 분석 도구 버그 수정

실험 결과 분석 스크립트(parse_results.py) 개발 과정에서 두 가지 심각한 파싱 버그가 발견되어 수정되었다.

11.1 버그 1: CSV,RX 오프셋 오류

CSV,RX 로그 형식은 CSV,RX,node=1,<src_ip>,<seq>,...이다. 기존 코드는 `parts[1].startswith("n")`를 검사하였으나 `parts[1] = "RX"`이므로 항상 False가 되어 offset이 0으로 고정되었다. 이후 `int("node=1")`에서 ValueError가 발생하여 **모든 RX 레코드가 무시되었고 모든 프로토콜의 PDR이 0.0으로 출력되었다.** 수정: `parts[2].startswith("node=")` 조건으로 변경.

11.2 버그 2: Seq-Only PDR 매칭

기존 PDR 계산: $|\{\text{tx seq}\} \cap \{\text{rx seq}\}| / |\{\text{tx seq}\}|$ 형태로 node ID 없이 seq 번호만 매칭하였다. 각 sender는 seq를 1부터 독립 카운팅하므로 seq 번호는 전역 유일하지 않다. 예: node 5의 seq=1과 node 7의 seq=1이 동일 집합 원소 {1}로 처리되어 **모든 프로토콜의 PDR이 1.0으로 과다 계산되었다.** 수정: `set(zip(node_id, seq))` 쌍 매칭으로 변경.

Table 7: 성분 절제 실험 결과 (5 seeds)

변형	활성 성분	Pre	During	Recovery
TA-BRPL (전체)	$T_f + T_c + T_h$	1.000	0.889	0.900
FWD only	T_f only	1.000	0.889	0.900
RPL (기준)	없음	1.000	0.877	0.875

Table 8: 링크 손실률별 during-attack PDR (5 seeds)

프로토콜	손실 0%	손실 10%	손실 20%
RPL	0.877	0.819 (-5.8 pp)	0.641 (-23.6 pp)
BRPL	0.880	0.836 (-4.4 pp)	0.696 (-18.4 pp)
TA-BRPL	0.889	0.874 (-1.5 pp)	0.697 (-19.2 pp)

12 전체 파라미터 목록

13 구현상 한계 및 향후 과제

L1 정적 토폴로지 결과 편중: 본 문서의 정량 결과는 고정 GRID 토폴로지에서 수행되었다. 이동성이 있는 환경에서는 잦은 parent 교체로 인해 에코 기반 T_{fwd} 카운터 축적이 불안정해질 수 있으며, PRR 폴백 비중이 증가할 것이다. 통제된 랜덤 토폴로지 생성/실행 프레임워크는 구현 완료되었으나, 대규모 통계 실행은 아직 수행하지 않았다.

L2 무손실 링크 가정: 기본 실험은 UDGM success_ratio=1.0 환경이다. 실제 IEEE 802.15.4에서 PRR $\approx 0.7-0.9$ 수준의 환경에서는 에코 기반 T_{fwd} 도 낮아지고 FP 위험이 증가한다. PRR 폴백이 ETX 기반 링크 품질을 반영하여 일부 완화하지만, 고손실 환경에서는 추가적인 링크 품질 보정이 필요하다.

L3 제한된 공격 유형: 현재 평가는 blackhole + rank sinkhole에 한정. Colluding sinkhole, version attack, Sybil attack, wormhole 등은 미평가.

L4 에코 의존성: T_{fwd} 의 에코 기반 측정은 하향 경로 (root $\rightarrow$ sender)가 개통되어야 한다. 에코가 도달하지 않는 구간(부트스트래핑 지연)에서는 PRR 폴백이 적용되지만 IP 계층 선택적 드롭을 탐지하지 못한다.

L5 부동소수점 의존성: 현재 Cooja 환경에서는 기하평균 계산에 glibc의 pow() 함수를 사용한다. 실제 MCU(Cortex-M0+ 등) 배포 시 정수 기반 lookup table로 대체가 필요하다.

L6 검증 모델 수렴 시간: 검증 모델은 하드 격리 전에 120초 이상 parent 안정화와 8회 이상 누적 전송을 요구하여 보수적이다. 단기 공격 시나리오에서는 충분한 증거 누적 전에 공격이 종료될 수 있다.

향후 과제:

- UART 로그 기반 에코 메커니즘 대신 Contiki-NG 내부 RX 이벤트 활용 검토.
- 정수 기반 기하평균 구현(lookup table).
- 실제 IEEE 802.15.4 테스트베드에서의 검증.
- 공모 공격(colluding attackers), version attack 환경 평가 확장.
- 30-seed 기준 통계적 유의성 검정 결과 상세 보고.

fig6: Parent Churn Hotspots (FWD-only vs Full TA-BRPL)

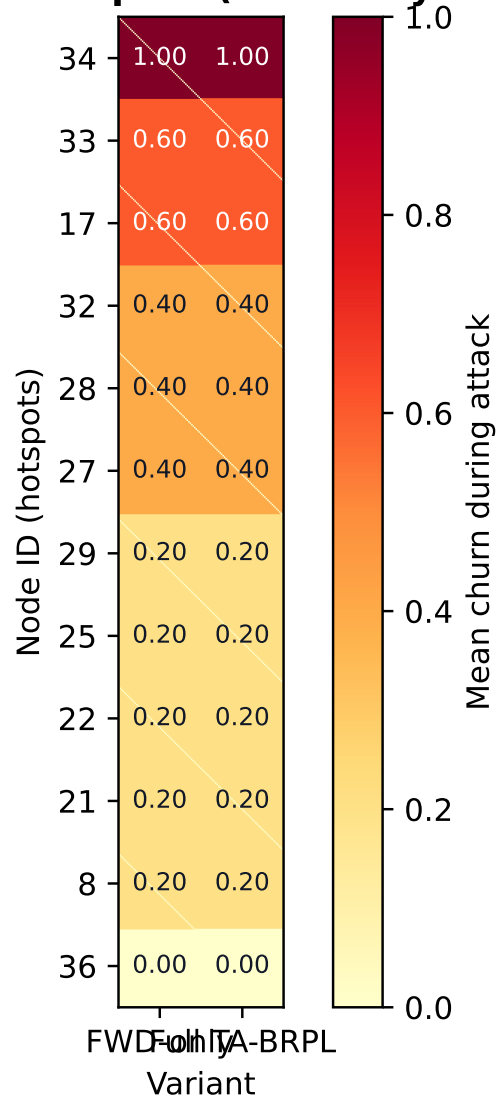


Figure 12: Parent churn 핫스팟 분포. FWD-only와 Full TA-BRPL의 parent 교체 패턴 비교.

14 결론

본 문서는 TA-BRPL의 전체 설계, 구현 상세, 동작 원리 및 실험 결과를 총정리하였다.

TA-BRPL은 다음의 핵심 기여를 제공한다.

1. **에코 기반 end-to-end 포워딩 신뢰 측정**: 802.15.4 MAC 필터링 제약을 우회하여 IP 계층 blackhole을 직접 탐지한다.
2. **PRR 폴백 부트스트래핑**: 에코 미수렴 구간에서의 trust 붕괴(death spiral)를 방지한다.
3. **2단계 검증 모델**: trust EWMA와 독립적인 누적 증거 기반으로 하드 격리 결정의 오탐을 억제한다.
4. **3성분 trust + 비대칭 EWMA**: 혼잡과 공격을 분리하고, on-off 공격 전략에 강인한 비대칭 EWMA를 적용한다.

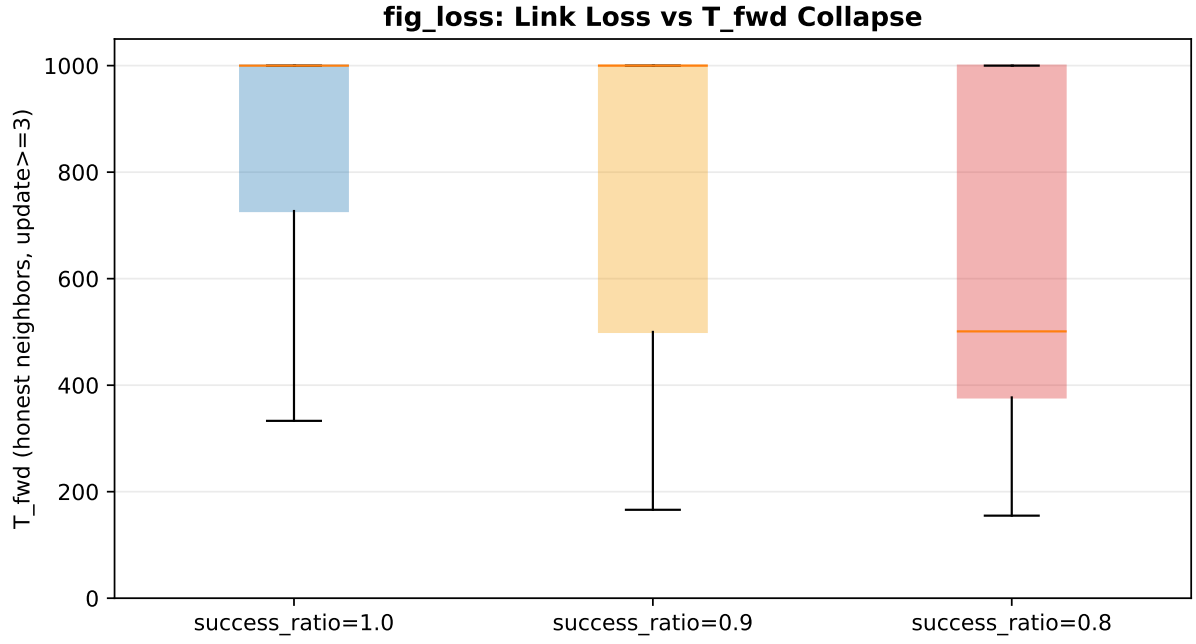


Figure 13: 손실률 증가에 따른 T_{fwd} 분포 변화. 손실이 커질수록 정상 노드와 공격자 간의 T_{fwd} 구별이 어려워진다.

5. **BRPL 통합**: weak-symbol 방식으로 기존 RPL-Classic 소스 수정 없이 결합하여 모듈성을 확보한다.

30-seed 실험에서 TA-BRPL은 during-attack PDR 0.8892, recovery PDR 0.8809를 달성하여 RPL, SMTrust 대비 우위를 확인하였으며, 무공격 환경에서 과오탐 없이 완전한 delivery를 유지하였다. 추가로, 통제된 랜덤 토폴로지 분포 평가를 위한 실험 인프라를 구축했으며, 후속 버전에서 해당 분포 기반 통계 결과를 통합 보고할 예정이다.

Table 9: TA-BRPL 주요 파라미터 (project-conf.h 기준)

파라미터	심벌/매크로	기본값
— 임계값 —		
Warn 임계값	τ_{warn} / TA_TRUST_TAU_WARN	600
Join 임계값	τ_{join} / TA_TRUST_TAU_JOIN	350
Blacklist 임계값	τ_{black} / TA_TRUST_TAU_BLACK	200
초기 신뢰	TA_TRUST_INIT	500
— EWMA —		
감소 람다 (집계)	$\lambda_{\downarrow}$ / TA_TRUST_LAMBDA_DECREASE	0.4
정상 람다	$\lambda_{\uparrow}$ / TA_TRUST_LAMBDA_NORMAL	0.7
감소 람다 (trust_fwd)	TA_TRUST_LAMBDA_DECREASE_FWD	0.2
업데이트 주기	TA_TRUST_UPDATE_INTERVAL	60s
— 집계 가중치 —		
T_{fwd} 가중치	w_f / TA_TRUST_W_FWD	5
T_{ctrl} 가중치	w_c / TA_TRUST_W_CTRL	3
T_{hon} 가중치	w_h / TA_TRUST_W_HON	2
— Laplace 스무딩 —		
Alpha	α / TA_TRUST_FWD_ALPHA	1
Beta	β / TA_TRUST_FWD_BETA	1
— T_{ctrl} 가중치 —		
Rank 이상	w_1 / TA_TRUST_CTRL_W_RANK	5
DIO 이상	w_2 / TA_TRUST_CTRL_W_DIO	3
Version 이상	w_3 / TA_TRUST_CTRL_W_VER	2
정상 DIO 비율	TA_TRUST_DIO_NORMAL_RATE	5
— Blacklist / 해제 —		
격리 기간	TA_TRUST_BLACKLIST_DURATION	120s
해제 후 복원 신뢰	TA_TRUST_RESTORE_ON_RELEASE	350
해제 쿨다운	TA_TRUST_RELEASE_COOLDOWN_SECONDS	120s
해제 초기 패널티 배율	TA_TRUST_RELEASE_PENALTY_SCALE_START	1600
— 지속 패널티 —		
기본 패널티 배율	TA_TRUST_ATTACK_PARENT_PENALTY_SCALE	1700
지속 창	TA_TRUST_ATTACK_PERSIST_WINDOW_SECONDS	120s
단계 추가	TA_TRUST_ATTACK_PERSIST_PENALTY_STEP	250
최대 패널티 배율	TA_TRUST_ATTACK_PERSIST_PENALTY_MAX	2600
— Escape —		
발동 시간	TA_TRUST_ESCAPE_TRIGGER_SECONDS	180s
패널티 배율	TA_TRUST_ESCAPE_PENALTY_SCALE	3200
escape 임계값	TA_TRUST_ESCAPE_TRUST_THRESHOLD	600
— 검증 모델 —		
부모 최소 나이 (활성화 조건)	TA_TRUST_VALIDATION_MIN_PARENT_AGE_SECONDS	120s
최소 전송 수 (창 활성화)	TA_TRUST_VALIDATION_MIN_SENT	3
누적 최소 전송 수	TA_TRUST_VALIDATION_MIN_ACC_SENT	8
bad 성공률 임계값	TA_TRUST_VALIDATION_BAD_SUCCESS_MAX	150
strong bad 성공률 임계값	TA_TRUST_VALIDATION_STRONG_SUCCESS_MAX	100
good 성공률 임계값	TA_TRUST_VALIDATION_GOOD_SUCCESS_MIN	550
bad 스코어 증가분	TA_TRUST_VALIDATION_BAD_INC	3
strong bad 보너스	TA_TRUST_VALIDATION_STRONG_BONUS	2
good 스코어 감소분	TA_TRUST_VALIDATION_GOOD_DEC	2
idle 스코어 감소분	TA_TRUST_VALIDATION_IDLE_DEC	1
UNDER 상태 임계 스코어	TA_TRUST_REVIEW_SCORE_UNDER	8
PENALIZED 상태 임계 스코어	TA_TRUST_REVIEW_SCORE_PENALIZED	14
blacklist 임계 스코어	TA_TRUST_REVIEW_SCORE_BLACKLIST	7